

# 10\_synthetic\_multiplicity

February 17, 2026

## 1 10 — Synthetic multiplicity dataset

**Purpose:** Train Rashomon models on a synthetic dataset with a known ambiguous region (island), compute predictive variance, Moran's I, and LISA; visualize variance and HH hotspots; report recovery (Precision/Recall/Jaccard) of the ground-truth island; fit a decision tree (HH vs non-HH); run the null experiment.

### 1.1 1. Imports and path setup

```
[27]: import sys
import importlib
from pathlib import Path

ROOT = Path.cwd().parent if Path.cwd().name == "notebooks" else Path.cwd()
if str(ROOT) not in sys.path:
    sys.path.insert(0, str(ROOT))
if str(ROOT / "src") not in sys.path:
    sys.path.insert(0, str(ROOT / "src"))

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from data import make_split, make_preprocessor
import src.synthetic_data
importlib.reload(src.synthetic_data)
from src.synthetic_data import (
    make_synthetic_multiplicity_dataset,
    make_synth_three_islands_plus_outliers,
    SyntheticGroundTruth,
    SyntheticGT,
)
from src.training_pipeline import run_one_training_run, save_run_artifacts
from analysis.run_analysis import (
    load_meta,
    load_P_test,
    select_rashomon_global,
    run_spatial,
```

```

    run_null,
    pointwise_variance,
    compute_multiplicity_metrics,
)

RESULTS_DIR = ROOT / "results"
K = 25
K_NN = 30
SEED = 42

```

## 1.2 2. Generate synthetic data

Stable region: two well-separated blobs. Ambiguous island: uniform disk around origin with weak XOR-like  $P(y=1|x)$ .

```

[28]: X, y, feature_info, gt = make_synthetic_multiplicity_dataset(
    n_samples=3000,
    p_island=0.2,
    island_delta=0.15,
    random_state=SEED,
)

print(f"Samples: {len(X)}, island (ambiguous): {gt.island_mask.sum()}, stable: {gt.stable_mask.sum()}")
print("Features:", feature_info)

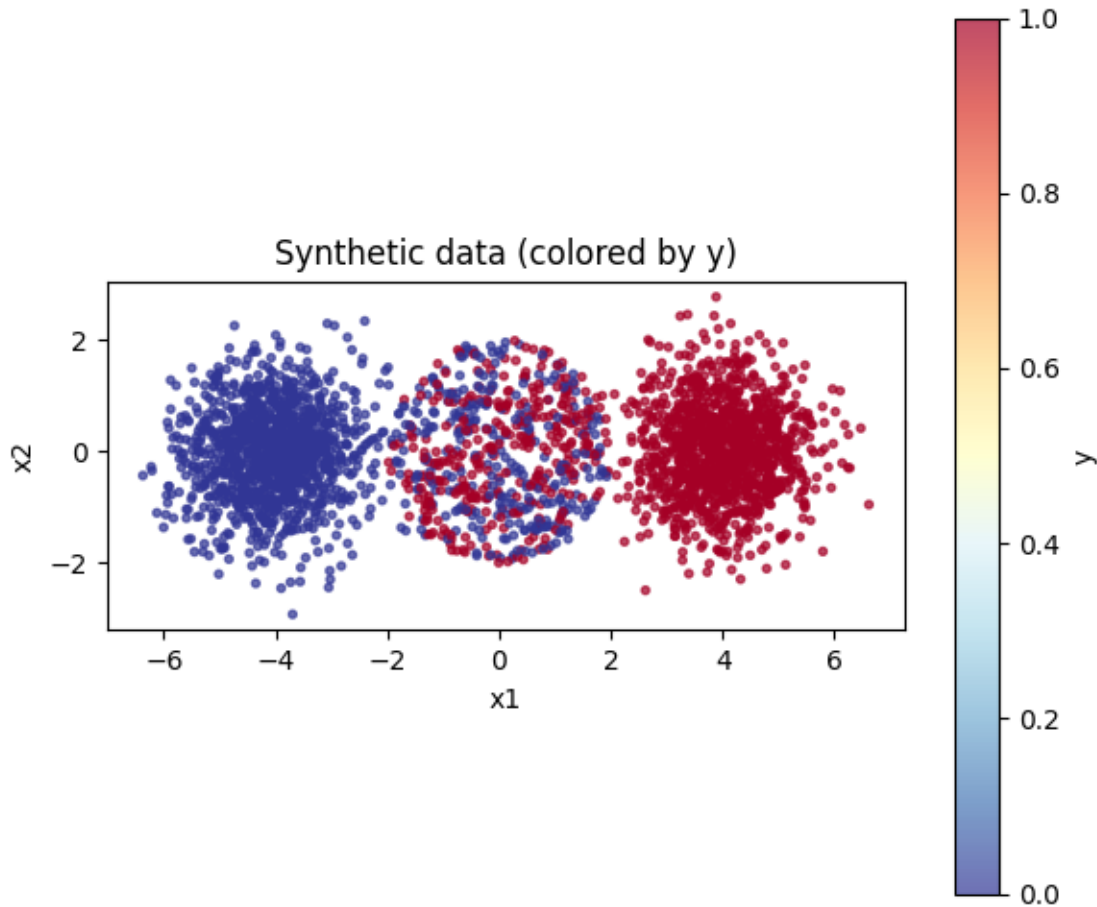
# Plot data colored by target y
plt.figure(figsize=(6, 5))
plt.scatter(X["x1"], X["x2"], c=y, s=8, alpha=0.7, cmap="RdYlBu_r")
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Synthetic data (colored by y)")
plt.colorbar(label="y")
plt.gca().set_aspect("equal")
plt.tight_layout()
plt.show()

```

```

Samples: 3000, island (ambiguous): 600, stable: 2400
Features: {'numeric': ['x1', 'x2'], 'categorical': []}

```



### 1.3 3. Train models and save artifacts

Single split (60/20/20), multiple families (LogReg, RF, GBM, MLP, kNN), top-K by validation Brier. Save to `results/synthetic/seed={SEED}/` so analysis functions can load from `run_dir`.

```
[29]: preprocessor_factory = lambda fi: make_preprocessor(fi, scale_numeric=True)

split, meta, P_val, P_test, _ = run_one_training_run(
    X=X,
    y=y,
    feature_info=feature_info,
    preprocessor_factory=preprocessor_factory,
    outer_seed=SEED,
    n_candidates_per_family=50,
    test_size=0.2,
    val_size=0.2,
    verbose=1,
)
```

```

run_dir = RESULTS_DIR / "synthetic" / f"seed={SEED}"
run_dir.mkdir(parents=True, exist_ok=True)
save_run_artifacts(run_dir, split, meta, P_val, P_test, config={"dataset": "
    ↪synthetic", "seed": SEED})

# Preprocessed X_test for spatial analysis (same as used inside pipelines)
preprocessor = preprocessor_factory(feature_info)
X_train = X.iloc[split["train"]]
y_train = y.iloc[split["train"]]
preprocessor.fit(X_train, y_train)
X_test_raw = X.iloc[split["test"]]
X_test = preprocessor.transform(X_test_raw)
if hasattr(X_test, "toarray"):
    X_test = np.asarray(X_test)
else:
    X_test = np.asarray(X_test)

print(f"Saved to {run_dir}")
print(f"Candidates: {len(meta)}, test size: {X_test.shape[0]}")

```

```

[LogReg] 50 candidates trained
[kNN] 50 candidates trained
[RF] 50 candidates trained
[GBM] 50 candidates trained
[MLP] 50 candidates trained
Saved to c:\Users\dejvi\Documents\pythonProject\Rashomon Sets\rashomon-
multiplicity\results\synthetic\seed=42
Candidates: 250, test size: 600

```

## 1.4 4. Rashomon selection and metrics

Select top-K by validation Brier; compute predictive variance, Moran's I, LISA (HH/LL masks).

```

[30]: n_cand = len(load_meta(run_dir))
K_actual = min(K, n_cand)
spatial = run_spatial(run_dir, X_test, K=K_actual, k=K_NN)
mult = ↪
    ↪compute_multiplicity_metrics(load_P_test(run_dir)[select_rashomon_global(run_dir, ↪
    ↪K=K_actual)], epsilon=0.05)

v = pointwise_variance(load_P_test(run_dir)[select_rashomon_global(run_dir, ↪
    ↪K=K_actual)], ddof=0)
HH_mask = spatial["HH_mask"]
LL_mask = spatial["LL_mask"]

print(f"Moran's I: {spatial['moran_i']:.4f}")
print(f"Mean variance: {mult['mean_variance']:.6f}")

```

```
print(f"HH count: {HH_mask.sum()}, LL count: {LL_mask.sum()}")
```

Moran's I: 0.4593

Mean variance: 0.001100

HH count: 101, LL count: 440

## 1.5 4b. Recovery metrics (HH vs ground-truth island)

Compare LISA HH hotspots to the ground-truth ambiguous region (island). Report **Precision**, **Recall**, and **Jaccard** as quantitative recovery metrics.

```
[31]: test_idx = split["test"]
island_test = gt.island_mask[test_idx] # ground-truth ambiguous region on test
      ↪ set

# HH_mask: predicted hotspots (LISA); island_test: true ambiguous region
tp = (HH_mask & island_test).sum()
pred_pos = HH_mask.sum()
true_pos = island_test.sum()

# Edge cases: no HH or no island
if pred_pos == 0:
    precision = np.nan
else:
    precision = tp / pred_pos
if true_pos == 0:
    recall = np.nan
else:
    recall = tp / true_pos

union = (HH_mask | island_test).sum()
if union == 0:
    jaccard = np.nan
else:
    jaccard = tp / union

print("Recovery of ground-truth island by LISA HH:")
print(f" Precision (HH island / HH): {precision:.4f}")
print(f" Recall (HH island / island): {recall:.4f}")
print(f" Jaccard (HH island / HH island): {jaccard:.4f}")
print(f" TP={int(tp)}, |HH|={int(pred_pos)}, |island|={int(true_pos)}")
```

Recovery of ground-truth island by LISA HH:

Precision (HH island / HH): 0.9901

Recall (HH island / island): 0.9091

Jaccard (HH island / HH island): 0.9009

TP=100, |HH|=101, |island|=110

## 1.6 5. Visualize predictive variance and HH points

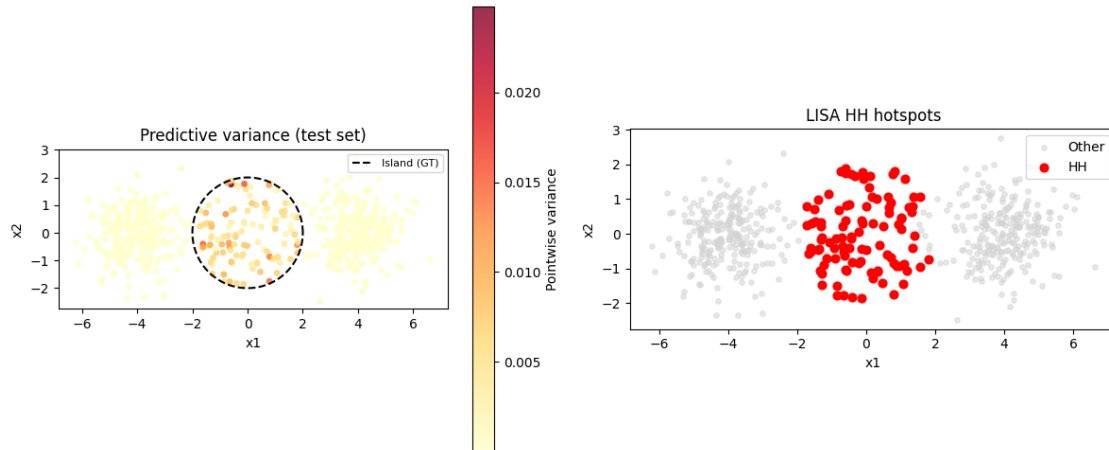
Test set in (x1, x2): color by pointwise variance; highlight HH hotspots. Optionally cluster and show variance / HH by cluster.

```
[32]: # Test set coordinates (original space for interpretability)
x1_test = X_test_raw["x1"].values
x2_test = X_test_raw["x2"].values
island_test = gt.island_mask[split["test"]] # ground-truth ambiguous region on
↳ test set

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Left: scatter colored by pointwise variance; overlay ground-truth island
↳ circle
sc = axes[0].scatter(x1_test, x2_test, c=v, s=15, cmap="YlOrRd", alpha=0.8)
theta = np.linspace(0, 2 * np.pi, 200)
axes[0].plot(gt.island_radius * np.cos(theta), gt.island_radius * np.
↳ sin(theta), "k--", lw=1.5, label="Island (GT)")
plt.colorbar(sc, ax=axes[0], label="Pointwise variance")
axes[0].set_xlabel("x1")
axes[0].set_ylabel("x2")
axes[0].set_title("Predictive variance (test set)")
axes[0].legend(loc="upper right", fontsize=8)
axes[0].set_aspect("equal")

# Right: HH points highlighted
axes[1].scatter(x1_test, x2_test, c="lightgray", s=15, alpha=0.5, label="Other")
axes[1].scatter(x1_test[HH_mask], x2_test[HH_mask], c="red", s=40, label="HH",
↳ zorder=5)
axes[1].set_xlabel("x1")
axes[1].set_ylabel("x2")
axes[1].set_title("LISA HH hotspots")
axes[1].legend()
axes[1].set_aspect("equal")
plt.tight_layout()
plt.show()
```



## 1.7 6. Decision tree: HH vs non-HH

Fit a decision tree on (x1, x2) to predict HH vs non-HH. Interpretable boundaries show which region the tree associates with LISA hotspots.

```
[33]: from sklearn.tree import DecisionTreeClassifier, plot_tree

X_test_2d = np.column_stack([x1_test, x2_test])
y_hh = HH_mask.astype(int)  # 1 = HH, 0 = non-HH

dt = DecisionTreeClassifier(max_depth=5, min_samples_leaf=10, random_state=SEED)
dt.fit(X_test_2d, y_hh)
acc = dt.score(X_test_2d, y_hh)
print(f"Decision tree (HH vs non-HH) accuracy on test set: {acc:.4f}")
print("Feature importances (x1, x2):", dt.feature_importances_)

# Plot tree
fig, ax = plt.subplots(figsize=(14, 8))
plot_tree(dt, feature_names=["x1", "x2"], class_names=["non-HH", "HH"],
    filled=True, ax=ax, fontsize=9)
plt.tight_layout()
plt.show()

# Scatter: true HH vs tree-predicted HH
pred_hh = dt.predict(X_test_2d)
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].scatter(x1_test, x2_test, c=y_hh, s=20, cmap="RdYlGn_r", alpha=0.8)
axes[0].set_xlabel("x1")
axes[0].set_ylabel("x2")
axes[0].set_title("True HH (green=non-HH, red=HH)")
axes[0].set_aspect("equal")
```

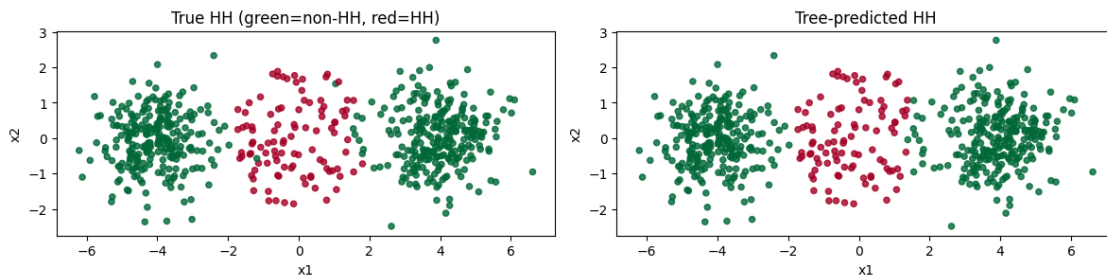
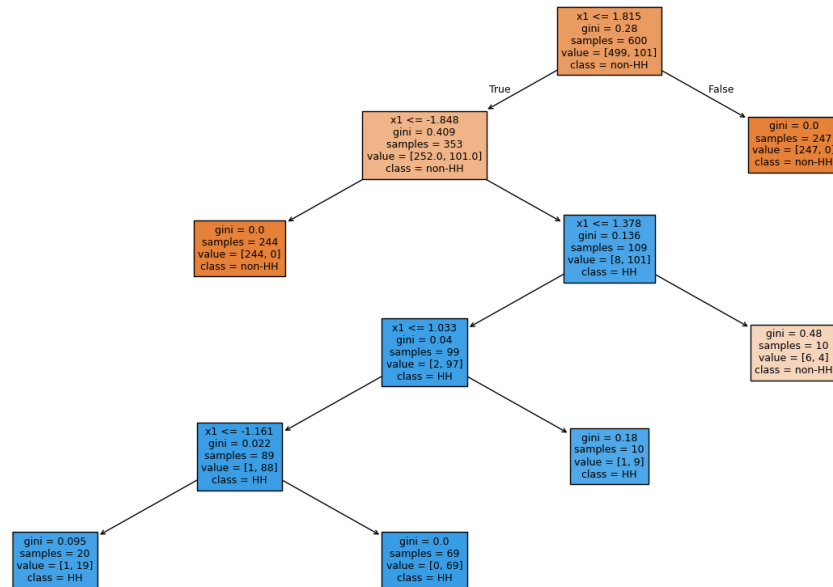
```

axes[1].scatter(x1_test, x2_test, c=pred_hh, s=20, cmap="RdYlGn_r", alpha=0.8)
axes[1].set_xlabel("x1")
axes[1].set_ylabel("x2")
axes[1].set_title("Tree-predicted HH")
axes[1].set_aspect("equal")
plt.tight_layout()
plt.show()

```

Decision tree (HH vs non-HH) accuracy on test set: 0.9900

Feature importances (x1, x2): [1. 0.]



## 1.8 7. Null experiment

Compare observed Moran's I to the null distribution (permuted predictions); plot histogram and empirical p-value.



```
[34]: R_null = 100
null_res = run_null(run_dir, X_test, K=K_actual, R=R_null, k=K_NN, seed=SEED)
observed_I = spatial["moran_i"]
null_moran = null_res["null_moran_i"]
p_emp = (1 + np.sum(null_moran >= observed_I)) / (R_null + 1)

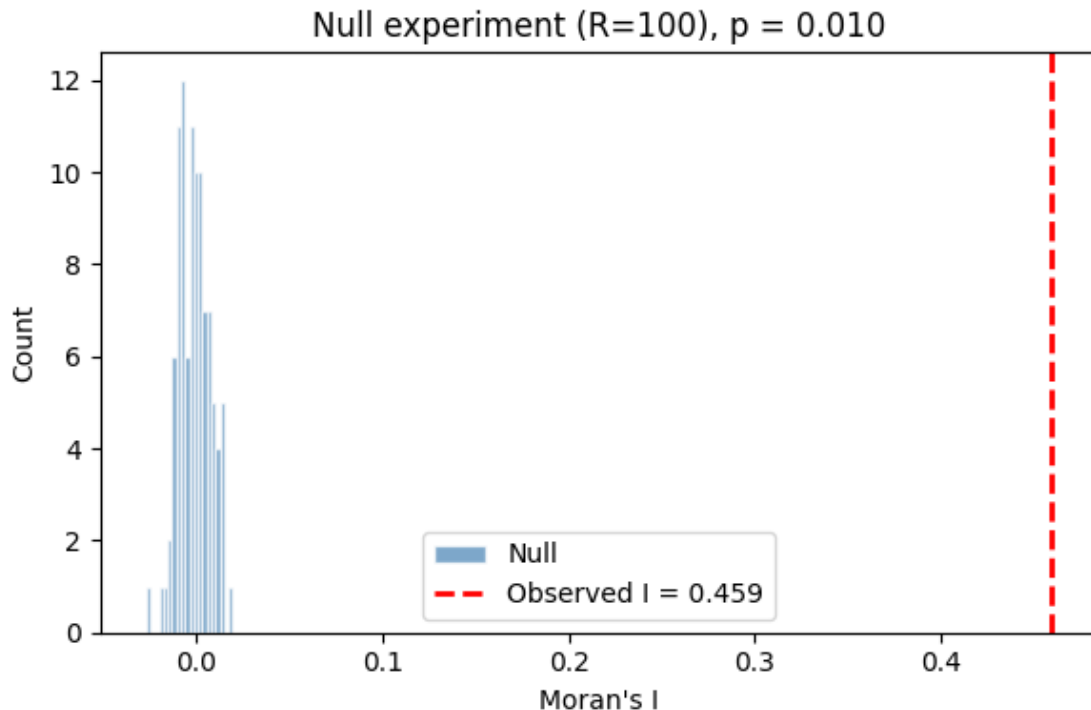
print(f"Observed Moran's I: {observed_I:.4f}")
print(f"Null mean ± std: {null_res['null_mean']:.4f} ± {null_res['null_std']:.4f}")
print(f"Empirical p-value (one-sided): {p_emp:.4f}")

plt.figure(figsize=(6, 4))
plt.hist(null_moran, bins=20, color="steelblue", alpha=0.7, edgecolor="white",
        label="Null")
plt.axvline(observed_I, color="red", linestyle="--", linewidth=2,
        label=f"Observed I = {observed_I:.3f}")
plt.xlabel("Moran's I")
plt.ylabel("Count")
plt.title(f"Null experiment (R={R_null}), p = {p_emp:.3f}")
plt.legend()
plt.tight_layout()
plt.show()
```

Observed Moran's I: 0.4593

Null mean ± std: -0.0013 ± 0.0084

Empirical p-value (one-sided): 0.0099



---

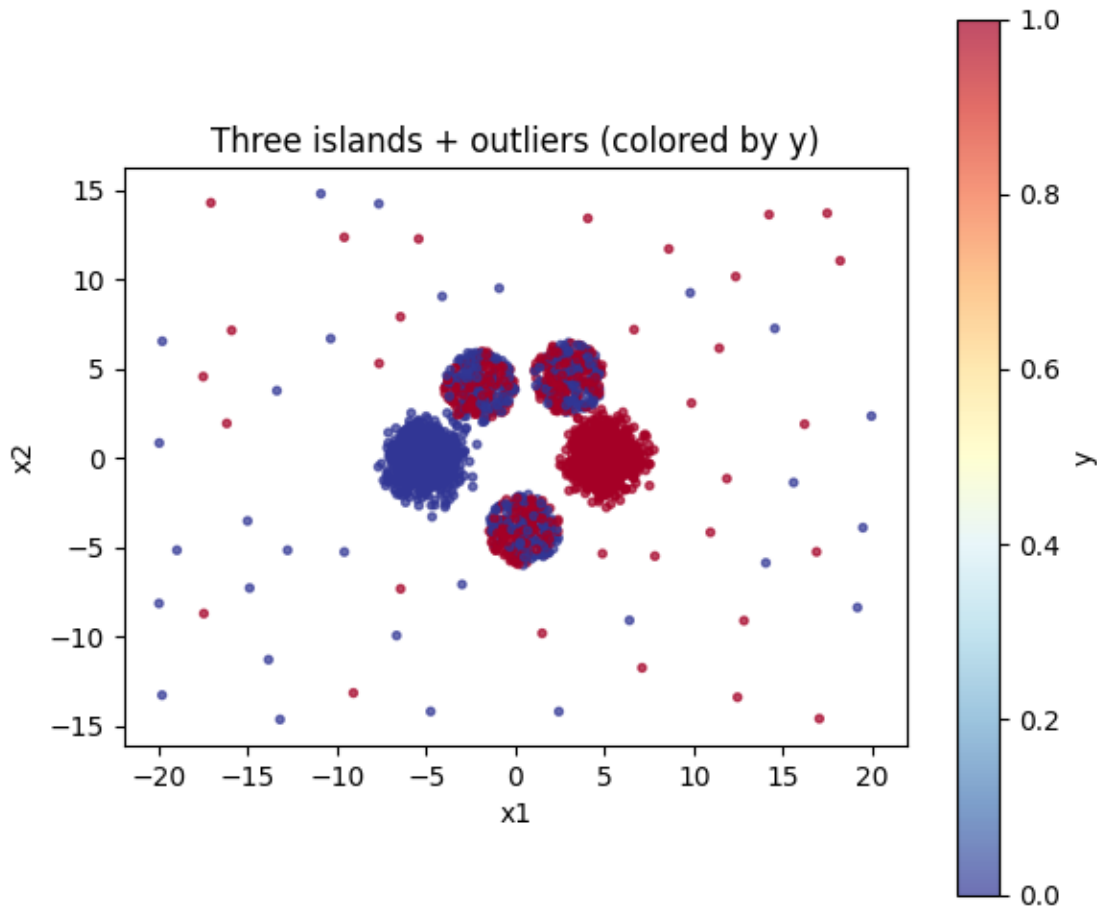
## 1.9 Part B: Three islands + outliers

Same pipeline on a second synthetic dataset: two stable blobs, **three** ambiguous islands, and isolated **outliers** (high-variance points that should not form HH clusters). Ground truth: `island_mask` (any of the 3 islands), `outlier_mask`, `stable_mask`.

```
[35]: # Dataset 2: three islands + outliers
X, y, feature_info, gt = make_synth_three_islands_plus_outliers(
    n_samples=3000,
    p_islands=0.30,
    p_outliers=0.02,
    island_delta=0.15,
    random_state=SEED,
)
print(f"Samples: {len(X)}, island: {gt.island_mask.sum()}, stable: {gt.
    ↪stable_mask.sum()}, outliers: {gt.outlier_mask.sum()}")

plt.figure(figsize=(6, 5))
plt.scatter(X["x1"], X["x2"], c=y, s=8, alpha=0.7, cmap="RdYlBu_r")
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Three islands + outliers (colored by y)")
plt.colorbar(label="y")
plt.gca().set_aspect("equal")
plt.tight_layout()
plt.show()
```

Samples: 3000, island: 900, stable: 2040, outliers: 60



```
[36]: # Train and save to a separate run dir
run_dir = RESULTS_DIR / "synthetic_three_islands" / f"seed={SEED}"
run_dir.mkdir(parents=True, exist_ok=True)

preprocessor_factory = lambda fi: make_preprocessor(fi, scale_numeric=True)
split, meta, P_val, P_test, _ = run_one_training_run(
    X=X, y=y, feature_info=feature_info,
    ↪preprocessor_factory=preprocessor_factory,
    outer_seed=SEED, n_candidates_per_family=50, test_size=0.2, val_size=0.2,
    ↪verbose=1,
)
save_run_artifacts(run_dir, split, meta, P_val, P_test, config={"dataset":
    ↪"synthetic_three_islands", "seed": SEED})

preprocessor = preprocessor_factory(feature_info)
preprocessor.fit(X.iloc[split["train"]], y.iloc[split["train"]])
X_test_raw = X.iloc[split["test"]]
```

```
X_test = np.asarray(preprocessor.transform(X_test_raw))
print(f"Saved to {run_dir}, test size: {X_test.shape[0]}")
```

```
[LogReg] 50 candidates trained
[kNN] 50 candidates trained
[RF] 50 candidates trained
[GBM] 50 candidates trained
[MLP] 50 candidates trained
Saved to c:\Users\dejvi\Documents\pythonProject\Rashomon Sets\rashomon-
multiplicity\results\synthetic_three_islands\seed=42, test size: 600
```

```
[37]: # Rashomon + spatial
n_cand = len(load_meta(run_dir))
K_actual = min(K, n_cand)
spatial = run_spatial(run_dir, X_test, K=K_actual, k=K_NN)
mult =
    ↳compute_multiplicity_metrics(load_P_test(run_dir)[select_rashomon_global(run_dir,
    ↳K=K_actual)], epsilon=0.05)
v = pointwise_variance(load_P_test(run_dir)[select_rashomon_global(run_dir,
    ↳K=K_actual)], ddof=0)
HH_mask = spatial["HH_mask"]
LL_mask = spatial["LL_mask"]
print(f"Moran's I: {spatial['moran_i']:.4f}, mean variance:
    ↳{mult['mean_variance']:.6f}, HH: {HH_mask.sum()}, LL: {LL_mask.sum()}")
```

```
Moran's I: 0.2887, mean variance: 0.001650, HH: 115, LL: 365
```

```
[38]: # Recovery: HH vs ground-truth island (any of the 3 islands)
test_idx = split["test"]
island_test = gt.island_mask[test_idx]
tp = (HH_mask & island_test).sum()
pred_pos = HH_mask.sum()
true_pos = island_test.sum()
precision = tp / pred_pos if pred_pos else np.nan
recall = tp / true_pos if true_pos else np.nan
union = (HH_mask | island_test).sum()
jaccard = tp / union if union else np.nan
print("Recovery of ground-truth islands by LISA HH:")
print(f" Precision: {precision:.4f}, Recall: {recall:.4f}, Jaccard: {jaccard:.
    ↳4f}")
print(f" TP={int(tp)}, |HH|={int(pred_pos)}, |island|={int(true_pos)}")
```

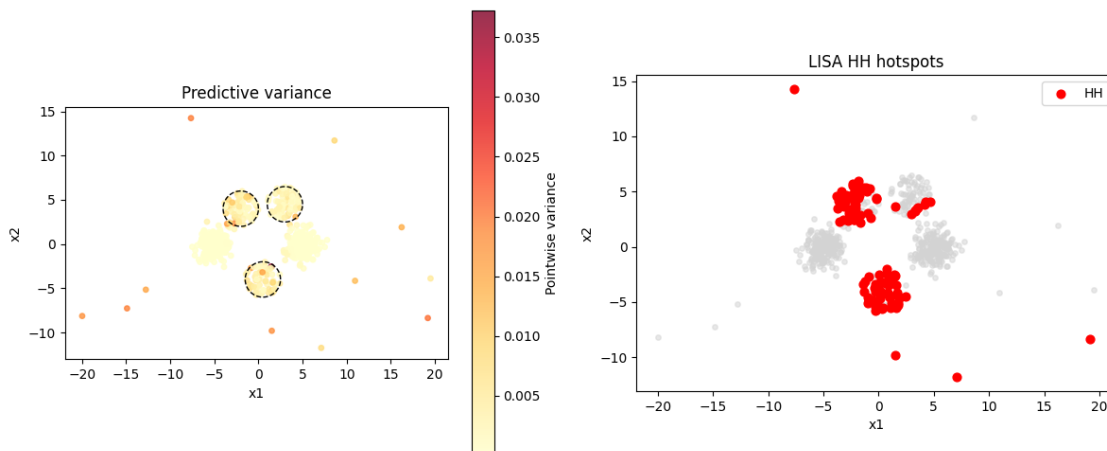
```
Recovery of ground-truth islands by LISA HH:
Precision: 0.9565, Recall: 0.5366, Jaccard: 0.5238
TP=110, |HH|=115, |island|=205
```

```
[39]: # Visualize variance and HH (three islands)
x1_test = X_test_raw["x1"].values
```

```

x2_test = X_test_raw["x2"].values
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
sc = axes[0].scatter(x1_test, x2_test, c=v, s=15, cmap="YlOrRd", alpha=0.8)
for c in gt.island_centers:
    theta = np.linspace(0, 2 * np.pi, 200)
    axes[0].plot(gt.island_radius * np.cos(theta) + c[0], gt.island_radius * np.
        ↪sin(theta) + c[1], "k--", lw=1)
plt.colorbar(sc, ax=axes[0], label="Pointwise variance")
axes[0].set_xlabel("x1"); axes[0].set_ylabel("x2"); axes[0].
    ↪set_title("Predictive variance"); axes[0].set_aspect("equal")
axes[1].scatter(x1_test, x2_test, c="lightgray", s=15, alpha=0.5)
axes[1].scatter(x1_test[HH_mask], x2_test[HH_mask], c="red", s=40, label="HH",
    ↪zorder=5)
axes[1].set_xlabel("x1"); axes[1].set_ylabel("x2"); axes[1].set_title("LISA HH
    ↪hotspots"); axes[1].legend(); axes[1].set_aspect("equal")
plt.tight_layout()
plt.show()

```

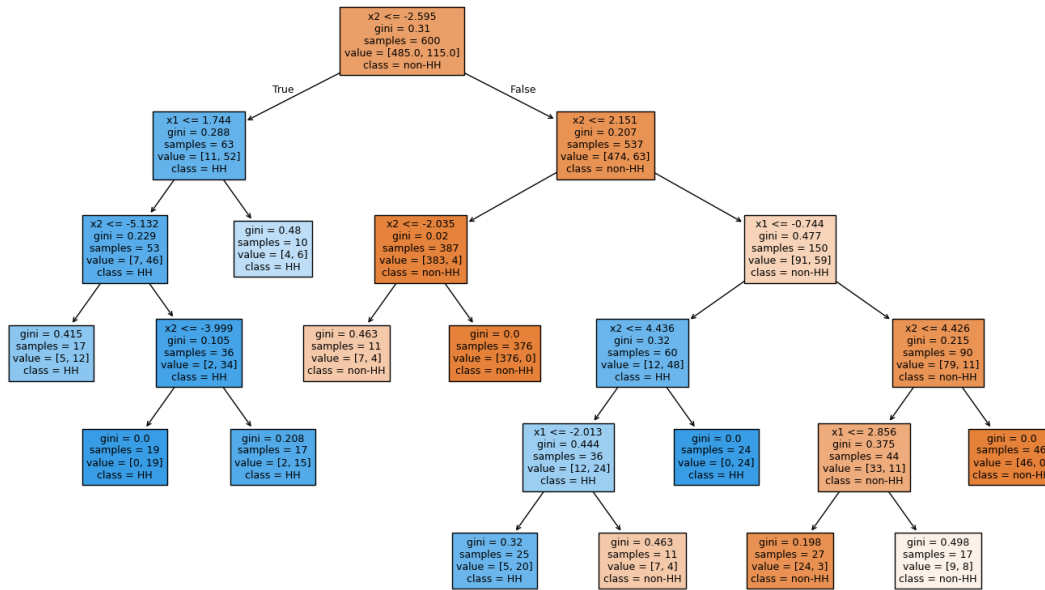


```

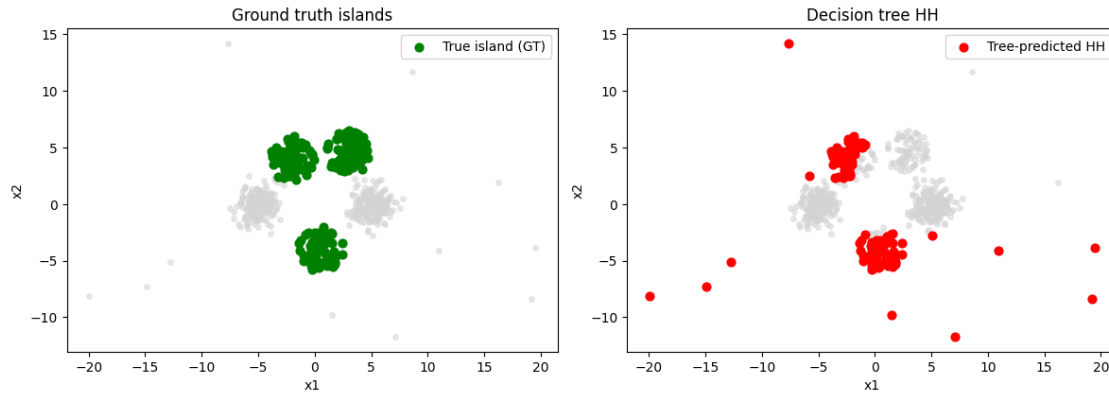
[40]: # Decision tree: HH vs non-HH (three islands)
from sklearn.tree import DecisionTreeClassifier, plot_tree
X_test_2d = np.column_stack([x1_test, x2_test])
y_hh = HH_mask.astype(int)
dt = DecisionTreeClassifier(max_depth=5, min_samples_leaf=10, random_state=SEED)
dt.fit(X_test_2d, y_hh)
print(f"Decision tree accuracy (HH vs non-HH): {dt.score(X_test_2d, y_hh):.4f}")
fig, ax = plt.subplots(figsize=(14, 8))
plot_tree(dt, feature_names=["x1", "x2"], class_names=["non-HH", "HH"],
    ↪filled=True, ax=ax, fontsize=9)
plt.tight_layout()
plt.show()

```

Decision tree accuracy (HH vs non-HH): 0.9417



```
[41]: # Scatter: true HH (island) vs tree-predicted HH (three islands)
hh_pred = dt.predict(X_test_2d).astype(bool)
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].scatter(x1_test, x2_test, c="lightgray", s=15, alpha=0.5)
axes[0].scatter(x1_test[island_test], x2_test[island_test], c="green", s=40,
    ↪label="True island (GT)", zorder=5)
axes[0].set_xlabel("x1"); axes[0].set_ylabel("x2"); axes[0].set_title("Ground_
    ↪truth islands"); axes[0].legend(); axes[0].set_aspect("equal")
axes[1].scatter(x1_test, x2_test, c="lightgray", s=15, alpha=0.5)
axes[1].scatter(x1_test[hh_pred], x2_test[hh_pred], c="red", s=40,
    ↪label="Tree-predicted HH", zorder=5)
axes[1].set_xlabel("x1"); axes[1].set_ylabel("x2"); axes[1].set_title("Decision_
    ↪tree HH"); axes[1].legend(); axes[1].set_aspect("equal")
plt.tight_layout()
plt.show()
```



```
[42]: # Null experiment (three islands)
null_res = run_null(run_dir, X_test, K=K_actual, R=100, k=K_NN, seed=SEED)
observed_I = spatial["moran_i"]
null_moran = null_res["null_moran_i"]
p_emp = (1 + np.sum(null_moran >= observed_I)) / 101
print(f"Observed Moran's I: {observed_I:.4f}, null mean±std:␣
      ↪{null_res['null_mean']:.4f} ± {null_res['null_std']:.4f}, p = {p_emp:.4f}")
plt.figure(figsize=(6, 4))
plt.hist(null_moran, bins=20, color="steelblue", alpha=0.7, edgecolor="white",␣
      ↪label="Null")
plt.axvline(observed_I, color="red", linestyle="--", linewidth=2,␣
      ↪label=f"Observed I = {observed_I:.3f}")
plt.xlabel("Moran's I"); plt.ylabel("Count"); plt.title(f"Null experiment␣
      ↪(three islands), p = {p_emp:.3f}"); plt.legend()
plt.tight_layout()
plt.show()
```

Observed Moran's I: 0.2887, null mean±std: -0.0029 ± 0.0081, p = 0.0099

